Experiment 'p_6_2_f_move_even_to_front_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_2_f_move_even_to_front_seq
Natural language statement: null
Method signature: p_6_2_f_move_even_to_front_seq(inputs: seq<int>) returns (result: seq<int>)

Ensures

- `result == evens(inputs) + odds(inputs)`

Functional Code Given

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
       then [s[0]] + evens(s[1..])
       else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
       then [s[0]] + odds(s[1..])
       else odds(s[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_6_2_f_move_even_to_front_seq(inputs: seq<int>) returns (result: seq<int>)

The method should respect the following contract:

ensures result == evens(inputs) + odds(inputs)

The contract uses the following dafny code:

function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
       then [s[0]] + evens(s[1..])
       else evens(s[1..])
}

function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
       then [s[0]] + odds(s[1..])
       else odds(s[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnNt9hHQydtoAc3iVeqX7Zl5Xjsj9
Model: gpt-5.5-2026-04-23
Created at: 1780662271
Finish reason: stop
Usage: 552 tokens in, and 3001 tokens out

```
{
```

```
  var i := |inputs|;
  var evenPart: seq<int> := [];
  var oddPart: seq<int> := [];

  assert inputs[|inputs|..] == [];
  assert evens(inputs[|inputs|..]) == [];
  assert odds(inputs[|inputs|..]) == [];

  while i > 0
    invariant 0 <= i <= |inputs|
    invariant evenPart == evens(inputs[i..])
    invariant oddPart == odds(inputs[i..])
    decreases i
  {
    i := i - 1;

    assert 0 <= i < |inputs|;
    assert |inputs[i..]| > 0;
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
      assert odds(inputs[i..]) == odds(inputs[i + 1..]);
      evenPart := [inputs[i]] + evenPart;
      assert evenPart == evens(inputs[i..]);
      assert oddPart == odds(inputs[i..]);
    } else {
      assert evens(inputs[i..]) == evens(inputs[i + 1..]);
      assert odds(inputs[i..]) == [inputs[i]] + odds(inputs[i + 1..]);
      oddPart := [inputs[i]] + oddPart;
      assert evenPart == evens(inputs[i..]);
      assert oddPart == odds(inputs[i..]);
    }
  }

  assert i == 0;
  result := evenPart + oddPart;
}
```

Final Program

```
function evens(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 == 0
       then [s[0]] + evens(s[1..])
       else evens(s[1..])
}
```

```
function odds(s: seq<int>): seq<int>
{
  if |s| == 0 then []
  else if s[0] % 2 != 0
       then [s[0]] + odds(s[1..])
       else odds(s[1..])
}

method p_6_2_f_move_even_to_front_seq(inputs: seq<int>) returns (result: seq<int>)
       ensures result == evens(inputs) + odds(inputs)
{
  var i := |inputs|;
  var evenPart: seq<int> := [];
  var oddPart: seq<int> := [];

  assert inputs[|inputs|..] == [];
  assert evens(inputs[|inputs|..]) == [];
  assert odds(inputs[|inputs|..]) == [];

  while i > 0
    invariant 0 <= i <= |inputs|
    invariant evenPart == evens(inputs[i..])
    invariant oddPart == odds(inputs[i..])
    decreases i
  {
    i := i - 1;

    assert 0 <= i < |inputs|;
    assert |inputs[i..]| > 0;
    assert inputs[i..][0] == inputs[i];
    assert inputs[i..][1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
      assert odds(inputs[i..]) == odds(inputs[i + 1..]);
      evenPart := [inputs[i]] + evenPart;
      assert evenPart == evens(inputs[i..]);
      assert oddPart == odds(inputs[i..]);
    } else {
      assert evens(inputs[i..]) == evens(inputs[i + 1..]);
      assert odds(inputs[i..]) == [inputs[i]] + odds(inputs[i + 1..]);
      oddPart := [inputs[i]] + oddPart;
      assert evenPart == evens(inputs[i..]);
      assert oddPart == odds(inputs[i..]);
    }
  }

  assert i == 0;
  result := evenPart + oddPart;
}
```

Total Token Usage

Input tokens: 552
Output tokens: 3001
Reasoning tokens: 2557
Sum of 'total tokens': 3553

Experiment Timings

Overall Experiment started at 1780662269584, ended at 1780662369548, lasting 99964ms (99.96 seconds)
Iteration #1 started at 1780662269584, ended at 1780662369548, lasting 99964ms (99.96 seconds)